SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKIH TEHNOLOGIJA OSIJEK

Sveučilišni prijediplomski studij Računarstva

APLIKACIJA ZA LOGIČKU IGRU MASTERMIND U PROGRAMSKOM JEZIKU C

Završni rad

Lara Lacić

Osijek, 2025.

SADRŽAJ

1. UVOD....3

1.1. Zadatak završnog rada....3

2. PREGLED PODRUČJA RADA....4

2.1. Genetski algoritam za rješavanje Mastermind-a....4

2.2. Složenost i problem zadovoljivosti....5

2.3. Postojeća programska rješenja za Mastermind....6

3. LOGIČKA IGRA MASTERMIND....9

3.1. Povijest Mastermind-a....9

3.2. Vrste Mastermind-a....10

3.2.1. Mini Mastermind....10

3.2.2. Super Mastermind....11

3.2.3. Word Mastermind....12

3.3. Number Mastermind....12

3.3.1. Postupak rješavanja....13

4. KORISNIČKO SUČELJE I KORIŠTENI ALATI....17

4.1. Programsko okruženje C....17

4.2. Microsoft Visual Studio....18

5. PROGRAMSKO RJEŠENJE....20

5.1. Varijable i konstante....20

5.2. Pomoćne funkcije....22

5.3. Glavni dio programa....26

5.4. Prikaz rezultata....30

6. ZAKLJUČAK....35

SAŽETAK....36

ABSTRACT....37

LITERATURA....38

1. UVOD

Mastermind je logička društvena igra u kojoj je cilj da tajni kod, zamišljen od strane drugog igrača ili računala, bude pogođen. Postoje različite vrste Mastermind-a, a u ovom radu razmatra se Number Mastermind, u kojem je potrebno pogoditi točan niz brojeva. Cilj igre ostvaruje se deduktivnim razmišljanjem i nizom pokušaja kojima se dolazi do traženog koda. Kroz konzolno sučelje omogućeno je igranje igre tako da se tajni niz simbola generira nasumično od strane računala, dok se pokušaji pogađanja ostvaruju unosom vlastitih kombinacija. Nakon svakog unosa prikazuju se povratne informacije o broju simbola koji su pogođeni na pravim pozicijama, kao i o broju simbola koji se nalaze u tajnom kodu, ali na različitim pozicijama.

U prvom poglavlju opisan je zadatak završnog rada. U drugom poglavlju opisani su osnovni pojmovi vezani za Mastermind i njegove vrste. Nadalje, objašnjen je algoritam rješavanja logičke zagonetke te programska rješenja dostupna na internetu. U trećem poglavlju opisan je Visual Studio i programski jezik C, korišten za implementaciju rješenja. U posljednjem poglavlju detaljno je prikazana implementacija programskog rješenja za Number Mastermind te dan primjer odigrane igre.

1.1. Zadatak završnog rada

Zadatak ovog završnog rada je izrada i objašnjenje programskog rješenja igre Mastermind u programskom jeziku C. Potrebno je implementirati validaciju unosa korisnika, ograničiti broj pokušaja, te jasno prikazati pravila igre i rezultate pokušaja.

2. PREGLED PODRUČJA RADA

U posljednjih dvadesetak godina razvijena su brojna računalna rješenja za igru Mastermind. Kao najpoznatija i dalje se ističe metoda Donalda Knutha (američkog informatičara, rođenog 1938.), kojom se za šifru sa četiri mjesta i šest boja jamči pronalazak rješenja u najviše pet poteza. Bit ideje sastoji se u tome da se svaki potez odabire prema algoritmu minimax (načelo "najgoreg slučaja"), čime se omogućuje brzo eliminiranje boja koje nisu prisutne u nizu. Na taj način potez se odabire tako da budu minimizirani maksimalni gubici, odnosno da se i u nepovoljnom ishodu dobije najmanji broj preostalih mogućih kombinacija.[1]

Također su se razvile informacijske metode koje sljedeći potez biraju prema tome koliko će smanjiti skup preostalih mogućih nizova. Najbolji potez je onaj koji dijeli preostale moguće kombinacije u manje i ravnomjernije podskupove, tako da se u idućim koracima broj mogućih kandidata smanjuje. Ovakav pristup, osobito kod duljih šifri i većeg broja boja, često brže vodi do rješenja jer svaka nova pretpostavka donosi maksimalnu količinu korisnih novih informacija. [2].

Drugi snažan smjer predstavljaju rješenja temeljena na programima kojima se rješava problem zadovoljivosti. U tom pristupu svaka dosadašnja pretpostavka i pripadajući odgovor prevode se u skup logičkih pravila kojima se opisuje što tajni kod ne smije sadržavati. Zatim se primjenjuju algoritmi za rješavanje logičkih formula (SAT) i maksimalne zadovoljivosti (MaxSAT). Na taj način kandidati u skladu s ograničenjima vrlo brzo su pronađeni. Nakon svake nove povratne informacije ograničenja se postupno nadopunjuju, a cijeli postupak ponavlja sve dok tajni kod ne bude pronađen. [3].

2.1. Genetski algoritam za rješavanje Mastermind-a

Jedan od pristupa rješavanju igre Mastermind temelji se na primjeni algoritama u kojima se genetski algoritam koristi kao sastavni dio pri traženju rješenja. Genetski algoritam predstavlja metodu optimizacije inspiriranu prirodnom evolucijom, pri čemu se koriste principi selekcije, križanja i mutacije kako bi najbolje rješenje bilo postupno pronađeno. U kontekstu Mastermind-a skupovi mogućih kombinacija generiraju se algoritmom, evaluiraju prema dosadašnjim povratnim informacijama te se kandidati za rješenje iterativno poboljšavaju. [4, 5]

Tijek algoritma odvija se u nekoliko faza. U početku se pretpostavlja da je P duljina koda koji se pokušava otkriti. Nakon što je zadan početni pokušaj, označen kao G_1, rezultat se označava u obliku dviju povratnih vrijednosti – X_1, koji predstavlja potpuno pogođene pozicije (tzv. "crveni čepovi"), i Y_1, koji označava djelomično točne pozicije, odnosno pogođenu boju na pogrešnom mjestu ("bijeli čepovi"). Ako broj potpuno pogođenih pozicija nije jednak duljini traženog koda P, algoritmom se ulazi u iterativni ciklus. [4, 5]

U svakoj novoj iteraciji inicijalizira se proces stvaranja nove populacije mogućih rješenja. To je omogućeno primjenom standardnih genetskih operatora: križanja (*crossover*), mutacije, inverzije i permutacije. Svaka kombinacija zatim se ocjenjuje pomoću funkcije prilagodbe (*engl. fitness function*), kojom se određuje koliko je pojedini niz blizu mogućem rješenju. Kombinacije koje zadovolje postavljene uvjete dodaju se u skup točnih rješenja. Na primjer, ako je unesen niz 1279 i povratnom informacijom vraćeno: „Jedna znamenka je pogođena, jedna znamenka je na krivom mjestu“, tada je dana ocjena koliko je kombinacija blizu rješenju. Nizovi koji dobiju najbolje ocjene (odnosno imaju najveći broj žutih i zelenih znamenki) pohranjuju se kao potencijalna rješenja. Novi pokušaj pogađanja koda odabire se tako da kombinacija koja je najvjerojatnije točna prema dosadašnjim rezultatima bude izabrana iz tog skupa. Cijeli postupak nastavlja se sve dok potpuna podudarnost s tajnom kombinacijom ne bude pronađena, čime se dolazi do rješenja i igra se završava. [4, 5]

2.2. Složenost i problem zadovoljivosti

Godine 2004. utvrđeno je od strane Michiela de Bondta (nizozemskog matematičara) da je problem rješavanja Mastermind-a putem računala iznimno zahtjevan, odnosno da se uvrštava u klasu NP (nedeterminističko polinomijalno vrijeme) kompletnih problema. NP-kompletni problem definira se kao problem koji je vrlo težak za riješiti, ali jednostavan za provjeru kada je rješenje već poznato. Ovo se odnosi na slučaj kada se igra odvija s n pozicija u svakom redu i samo dvije različite boje. U radu Mastermind je povezan s problemom one-in-three 3SAT, poznatim NP-kompletnim problemom iz teorije računarstva i logike. U one-in-three 3SAT svaka klauzula sadrži tri varijable, a uvjet je da točno jedna od njih bude istinita. Na sličan način, svaki red u Mastermind igri s pokušajem i dobivenom povratnom informacijom predstavlja uvjet koji tajni kod mora zadovoljiti. Pokazano je da se svaka klauzula može preslikati u jedan red Mastermind igre, pri čemu povratne informacije određuju koji rasporedi boja ostaju mogući. [4, 6, 7, 8]

Na primjer, u slučaju pogađanja tajnog niza od tri znamenke u kojem se mogu unositi samo brojevi 0 i 1, ako je unesen niz 111 i povratnom informacijom dano da je jedna znamenka na točnom mjestu, zaključuje se da se u traženom nizu nalazi jedna znamenka 1, dok ostale moraju biti 0. Na temelju toga može se utvrditi da su jedine moguće kombinacije koje zadovoljavaju ovaj uvjet 001, 010 ili 100. Upravo se na ovome uočava poveznica s problemom *one-in-three 3SAT*: svaki pokušaj zajedno s dobivenom povratnom informacijom predstavlja pravilo koje tajni niz mora zadovoljiti. Kada se pravila objedine iz više pokušaja, zajedno čine skup ograničenja kojim se određuje koje kombinacije još mogu biti točan odgovor. Time je dokazana ekvivalencija između rješavanja logičkih formula i Mastermind igre, čime je potvrđeno da Mastermind predstavlja računalno kompleksan izazov, čak i u vrlo jednostavnim varijantama s ograničenim brojem boja.[4, 6, 7, 8]

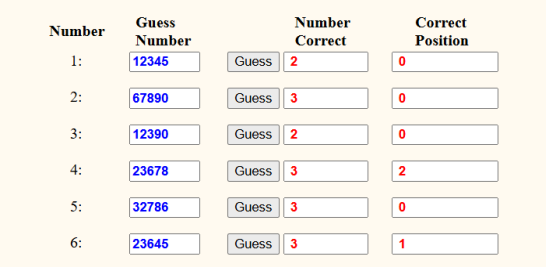
Istražen je i tzv. Dosljedni Mastermind, verzija igre u kojoj svaki novi pokušaj mora ovisiti o svim prethodno primljenim povratnim informacijama. Ovim principom rješavanja postavljeno je pravilo da nasumične kombinacije ne smiju biti unesene, već se svaki sljedeći pokušaj mora temeljiti na rezultatima ranijih pokušaja. Na primjer, ako je u prvom pokušaju unesen broj koji se ne pojavljuje nigdje unutar niza, taj broj u svim kasnijim pokušajima ne smije biti korišten. Ako je unesen broj koji jest dio niza, ali nije na ispravnom mjestu, taj broj mora biti korišten sve dok njegova točna pozicija ne bude pronađena. Ovim načinom dodatno se povećava složenost igre jer se informacije moraju kombinirati i logički povezivati kako bi se odredile kombinacije koje još uvijek mogu predstavljati tajni niz.[4]

Ova analiza uvodi u problem poznat kao *Mastermind Satisfiability Problem* (dalje u tekstu MSP). Osnovna hipoteza MSP-a glasi: ako je poznat niz pokušaja te broj crvenih i bijelih čepova, odnosno potpune i djelomične podudarnosti za svaki pokušaj, postavlja se pitanje postoji li uopće neka tajna kombinacija kojom bi mogle biti generirane upravo takve povratne informacije. Ovaj problem svodi se na provjeru može li uopće postojati ispravno rješenje za zadani skup podataka. Ukoliko nijedna kombinacija ne zadovoljava uvjete, znači da je pogrešna informacija vraćena barem jednom od svih mogućih pokušaja, što dodatno pokazuje koliko igra može biti složena u teorijskom smislu. [4, 9]

2.3. Postojeća programska rješenja za Mastermind

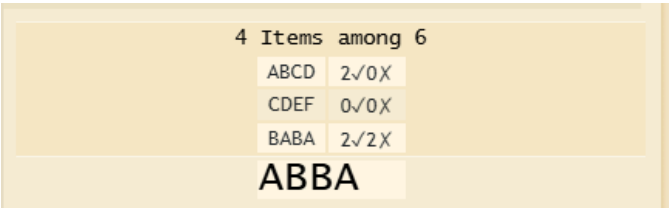
Na internetu je dostupan velik broj različitih rješenja igre Number Mastermind. Najčešća internetska rješenja temelje se na unosu pokušaja od strane korisnika, nakon čega se prikazuje

koliko je znamenki pogođeno u potpunosti, a koliko ih se nalazi na pogrešnom mjestu. Primjer takvog rješenja nalazi se na stranici Easysurf, gdje se pet različitih znamenki nasumično odabire računalom, a kombinacija se pokušava pogoditi u najviše dvadeset pokušaja. Za razliku od ovog rješenja, u programu izrađenom u sklopu ovog rada duljina niza odabire se od strane korisnika, dopuštena su ponavljanja znamenki i igra je vremenski ograničena. Prikaz sučelja igre dan je na slici 2.1. [10]



Slika 2.1 : Sučelje igre na stranici Easysurf

Drugi poznati primjer predstavlja *dCode.fr Mastermind Solver*, alat kojim je omogućeno da korisnik unese vlastite pokušaje i pripadajuće povratne informacije, nakon čega se računalno pronalazi moguće rješenje. U programu izgrađenom u ovom radu tajni niz generira se automatski, a povratne informacije određuju se samostalno, dok su pokušaji uneseni od strane korisnika. Prikaz sučelja igre dan je na slici 2.2. [11]



Slika 2.2 : Sučelje igre na stranici dCode

Osim mrežnih verzija, razvijene su i mobilne aplikacije poput *Numerin Mastermind* koje nude dodatne funkcionalnosti, među kojima se nalaze vizualne oznake, dnevni izazovi te mogućnost igranja protiv drugih korisnika. Na platformama GitLab i GitHub dostupna je nekolicina programskih rješenja za igru Number Mastermind. Većina tih projekata usmjerena je na automatsko rješavanje igre, bez potrebe za unosom od strane korisnika ili grafičkim sučeljem.

Kao primjeri mogu se navesti algoritmi kojima se generiraju sve moguće kombinacije, analiziraju dobivene povratne informacije i odabiru optimalni pokušaji kako bi rješenje bilo pronađeno u što manjem broju koraka. Neki od projekata su:

- Mastermind Numbers Solver [12]
- mastermind-solver [13]
- Mastermind-game [14]

Glavna razlika između ovih mrežnih rješenja i programa izrađenog u sklopu ovog rada nalazi se u načinu implementacije i sučelju. Internetske aplikacije i mobilne igre razvijene su korištenjem jezika poput JavaScripta, HTML-a, CSS-a ili tehnologija za razvoj mobilnih aplikacija, a karakterizira ih grafičko i interaktivno sučelje. Za razliku od njih, program izrađen u ovom radu napisan je u programskom jeziku C, pokreće se isključivo u konzoli i ima vremensko ograničenje. Grafičko sučelje nije uključeno, već se igra temelji na unosu podataka putem tipkovnice i tekstualnom prikazu povratnih informacija.

3. LOGIČKA IGRA MASTERMIND

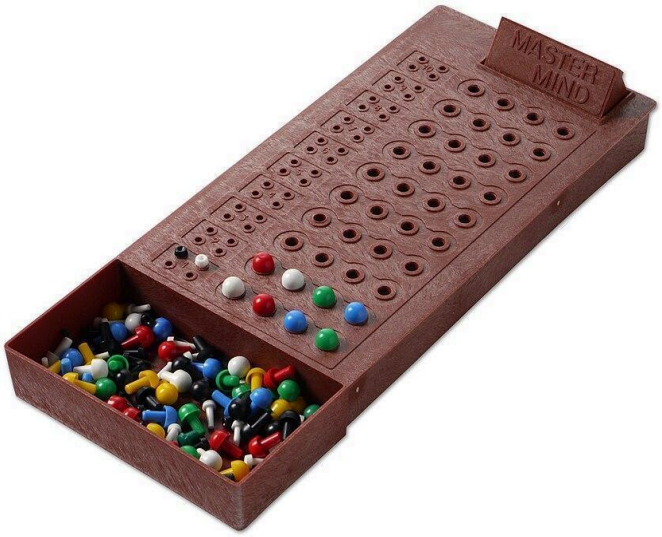
Mastermind je logička društvena igra kojom se potiče deduktivno razmišljanje i analitičko zaključivanje. U igri sudjeluju dva igrača, pri čemu je jedan zadužen za stvaranje tajnog koda, dok je drugi zadužen za pogađanje točne kombinacije. Kod se sastoji od četiri boje poredane u nizu, a kombinacija se pokušava pogoditi odabirom između šest ponuđenih boja. Nakon svakog pokušaja daje se povratna informacija pomoću malih čepova: crvenim čepom označava se da se određena boja nalazi na točnom mjestu, dok bijelim čepom označava se da se boja nalazi u kodu, ali na drugoj poziciji. Na temelju ovih podataka dolazi se do rješenja. Cilj igre je da skrivena kombinacija bude pogođena u što manjem broju pokušaja. Primjer točno riješene igre prikazan je na slici 3.1.. [4]



Slika 3.1 : Riješena igra Mastermind

3.1. Povijest Mastermind-a

Mastermind je osmišljen od strane izraelskog izumitelja Mordecaija Meirowitza (izraelskog stručnjaka za telekomunikacije i proizvođača igara, rođenog 1930. godine u Rumunjskoj). Nekoliko izdavača igru je u početku odbilo, no njezin potencijal uočen je od strane britanske tvrtke *Invicta Plastics*, koja ju je izdala 1970. godine. Vrlo brzo igra je stekla svjetsku popularnost, postala globalni hit i počela biti distribuirana u više od 80 zemalja. Kao inspiracija za igru poslužila je starija papirnata igra poznata kao “Bulls and Cows”. Obje igre temelje se na istom principu, odnosno deduktivnom zaključivanju. Originalni set društvene igre iz tog razdoblja prikazan je na slici 3.2. [4]



Slika 3.2 : Originalni set igre Invicta

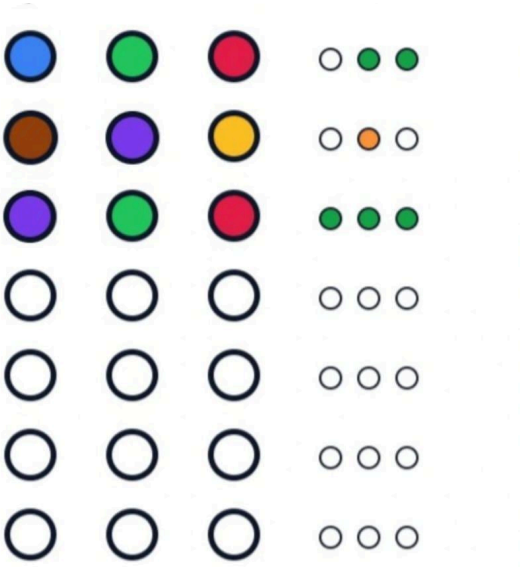
Već 1960-ih godina Mastermind je po prvi puta adaptiran za računalo. Na računalnom sustavu Titan Sveučilišta u Cambridgeu igra je bila poznata pod nazivom “MOO”. Kasnije su razvijene brojne druge inačice igre za različite računalne sustave. [4]

3.2. Vrste Mastermind-a

Postoji više vrsta igre Mastermind koje se razlikuju po vrsti i broju znakova koji se pogađaju i načinu prikaza rezultata. Osim klasične verzije s bojama, danas postoje brojne digitalne verzije igre koje koriste brojeve, slova ili riječi. U nastavku su opisane najpoznatije vrste: Mini Mastermind**,** Super Mastermind**,** Word Mastermind i Number Mastermind. [4]

3.2.1. Mini Mastermind

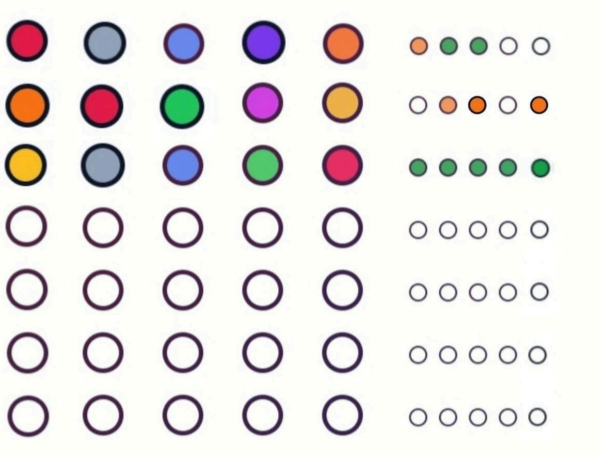
Mini Mastermind (slika 3.3.) predstavlja pojednostavljenu i manju verziju klasične igre, u kojoj se koristi kraći niz, najčešće s tri pozicije umjesto četiri, te ograničen broj boja (najčešće šest). Zbog manjih dimenzija i lakšeg sučelja, ova verzija smatra se idealnom za mlađe igrače, ali i za putovanja. Iako jednostavnija, u Mini Mastermind verziji zadržani su osnovni principi logičkog zaključivanja i strategije, pri čemu se nudi brza, ali izazovna varijanta igre. [4]



Slika 3.3 : Riješena igra Mini Mastermind

3.2.2. Super Mastermind

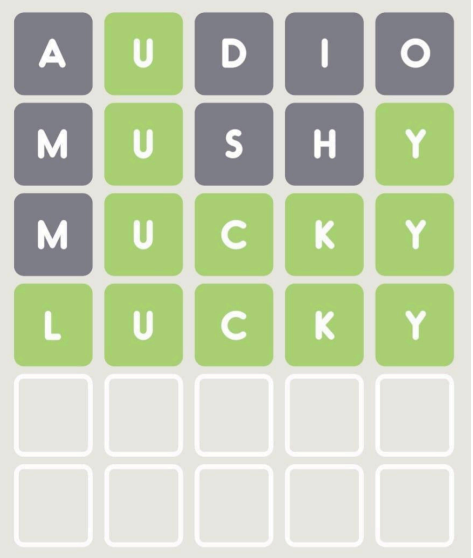
Super Mastermind (slika 3.4.) predstavlja proširenu verziju klasične igre, koja se razlikuje po duljem kodu te većem broju mogućih boja. Ovako proširena igra značajno povećava broj mogućih kombinacija, čime se igra čini znatno zahtjevnijom. Upravo zbog toga Super Mastermind smatra se verzijom namijenjenom iskusnijim igračima koji traže složenije logičke izazove. Uz veći broj varijabli omogućeno je dublje promišljanje i pruženo naprednije iskustvo igre. [4]



Slika 3.4 : Riješena igra Super Mastermind

3.2.3. Word Mastermind

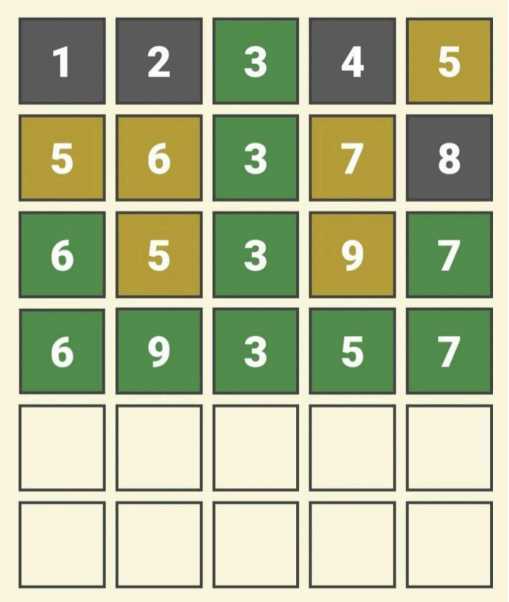
Word Mastermind (slika 3.5.) zamjenjuje tradicionalne šarene čepove slovima, čime se igra pretvara u jezičnu zagonetku. Nakon svakog pokušaja daje se povratna informacija: slova koja su na ispravnoj poziciji prikazuju se u zelenoj boji, dok se ona koja pripadaju riječi, ali su na pogrešnom mjestu, označavaju narančasto. U ovoj verziji najčešće se koriste riječi duljine četiri do pet slova, no igra se može lako prilagoditi različitim razinama težine. Word Mastermind smatra se idealnim edukativnim alatom jer se njime potiče razvoj vokabulara, prepoznavanje obrazaca i logičko razmišljanje, čime se omogućuje koristan i zabavan način učenja jezika.[4]



Slika 3.5 : Riješena igra Word Mastermind

3.3. Number Mastermind

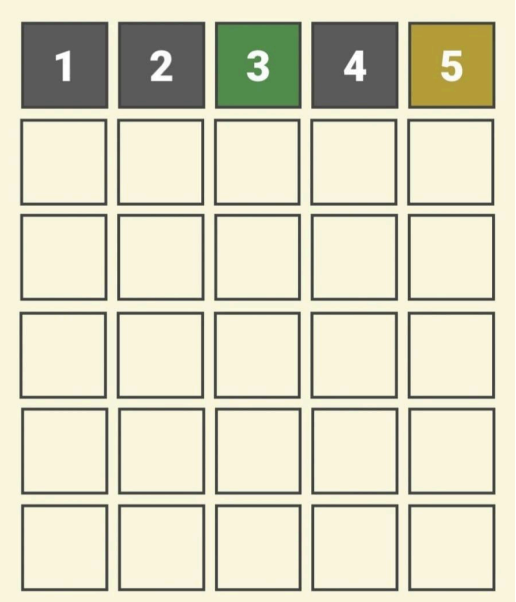
Number Mastermind (slika 3.6.) predstavlja verziju igre u kojoj su boje zamijenjene brojevima, pri čemu se koriste znamenke od 0 do 9. U nekim varijacijama uključene su dodatne informacije poput zbroja svih znamenki, čime je povećana složenost igre i dodana dodatna logička dimenzija. Ova inačica osobito se smatra zanimljivom za ljubitelje matematike i one koji vole izazovne brojčane zagonetke.[4]



Slika 3.6 : Riješena igra Number Mastermind

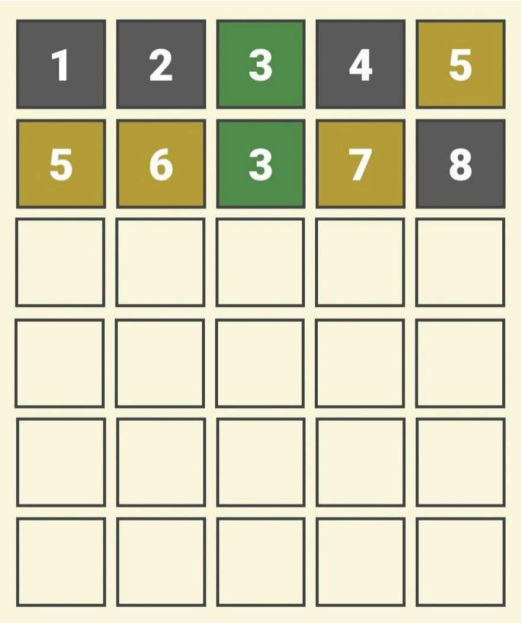
3.3.1. Postupak rješavanja

Postupak rješavanja Number Mastermind-a sa slike 3.6. prikazan je slikama 3.7. do 3.10. Prvi korak jednak je kao i u svakoj vrsti Mastermind igre – proizvoljno se odabire kombinacija od pet znamenki koja se unosi kao prvi pokušaj.



Slika 3.7 : Prvi korak rješavanja

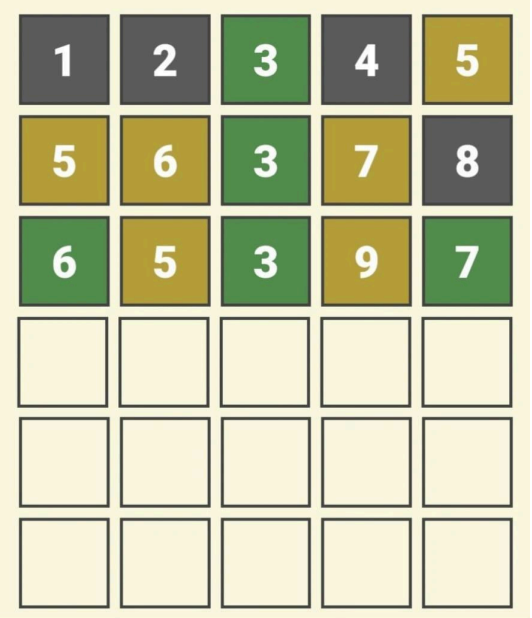
Nakon što je unesen niz znamenki, vraća se vizualna povratna informacija u obliku boja kojima se označava točnost svake znamenke. Ako je neka znamenka označena zelenom bojom, time je naznačeno da se ta znamenka nalazi na ispravnoj poziciji u tajnom nizu. U danom primjeru znamenka na trećem mjestu označena je zelenom bojom, što znači da se upravo ona nalazi na tom mjestu u rješenju. Žuta boja, poput one kojom je označena znamenka 5, ukazuje na to da se ta znamenka nalazi negdje u tajnom nizu, ali ne na trenutno upisanom mjestu, pa se u sljedećem pokušaju premješta na drugu poziciju. Znamenke označene crnom bojom nisu dio tajnog niza te se u sljedećem unosu zamjenjuju novim, još neiskorištenim znamenkama, kako bi se provjerilo nalaze li se one možda u rješenju.



Slika 3.8 : Drugi korak rješavanja

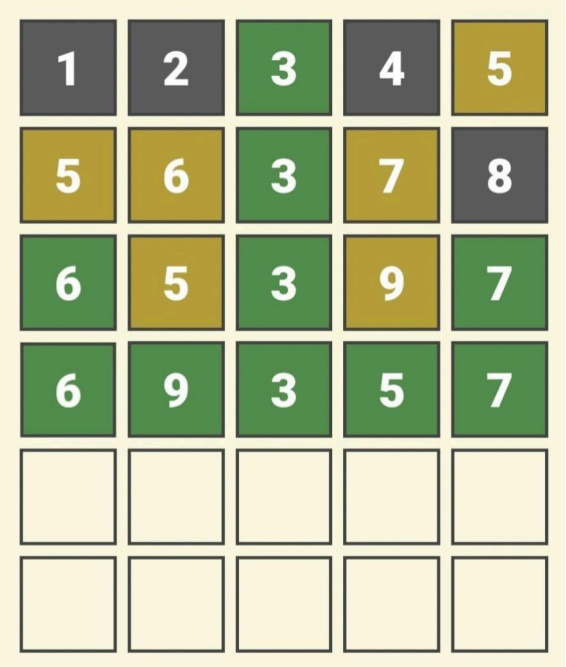
Nakon drugog pokušaja primjenjuje se ista logika kao i u prethodnom koraku kako bi novi podaci bili analizirani. Uočava se da se znamenka 5 i dalje ne nalazi na ispravnom mjestu, ali budući da je već testirana na prvoj i petoj poziciji bez uspjeha, zaključuje se kako njezina točna pozicija može biti jedino druga ili četvrta. Time se sužavaju mogućnosti i povećava vjerojatnost pronalaska točne kombinacije. Osim toga, znamenke 6 i 7 označene su žutom bojom, što znači da su dio rješenja, ali na pogrešnim pozicijama, pa se sada pokušavaju smjestiti na preostala slobodna mjesta u nizu kako bi se testiralo gdje točno pripadaju. Znamenka 8 označena je crnom bojom, čime je naznačeno da nije dio tajnog niza, pa se zamjenjuje nekom od preostalih

znamenki koje još nisu korištene, kako bi se nastavio postupak eliminacije i traženja ispravne kombinacije.



Slika 3.9 : Treći korak rješavanja

U posljednjem koraku preostaje još samo zamijeniti mjesta znamenkama 5 i 9, budući da su obje već potvrđene kao dio tajnog niza, ali nisu na svojim ispravnim pozicijama. Analizom prethodnih pokušaja zaključeno je da se one nalaze na suprotnim mjestima. Ostale znamenke već su pravilno postavljene, pa nisu mijenjane. Budući da su druga i četvrta pozicija jedine preostale za razmjenu, znamenke 5 i 9 smještene su upravo ondje. Nakon ove zamjene cijeli niz označen je zelenom bojom, čime je naznačeno da se sve znamenke nalaze na točnim pozicijama. Time je rješenje pronađeno, a igra uspješno završena.



Slika 3.10 : Završena igra

4. KORISNIČKO SUČELJE I KORIŠTENI ALATI

4.1. Programsko okruženje C

Programski jezik C razvijen je u Bell Labs-u 1972. godine. Razvijen je od strane Dennisa Ritchiea s ciljem da se omogući razvoj UNIX operacijskog sustava. C jezik pripada proceduralnim i imperativnim jezicima, a njegova je struktura oblikovana tako da omogućuje jasno i učinkovito programiranje. Tijekom godina, C jezik postao je temelj za mnoge druge jezike, uključujući C++, C#, Javu i brojne druge, čime je potvrđena njegova iznimna važnost u povijesti računarstva.

Najveća prednost C-a ogleda se u brzini izvođenja programa, budući da se kod prevodi izravno u strojni kod bez dodatnih slojeva interpretacije. Zbog toga se programi napisani u C-u smatraju iznimno učinkovitim i pogodnim za razvoj softvera koji zahtijeva visoke performanse. Pomoću pokazivača omogućen je direktan pristup memoriji, čime se programeru pruža velika fleksibilnost, ali istodobno i odgovornost za pravilno upravljanje resursima. C karakterizira mali broj ključnih riječi i minimalna standardna biblioteka, zbog čega se moraju strogo pratiti pravila jezika. Upravo ta jednostavnost u sintaksi, uz istodobnu moć, pridonijela je njegovoj dugovječnosti i popularnosti.

Osnovni pojmovi vezani za C uključuju definiranje osnovnih varijabli i tipova podataka poput *int*, *char*, *float* i *double*, zatim upotrebu kontrolnih struktura kao što su *if*, *for*, *while* i *switch*. Osim toga, jezik sadrži bogat skup funkcija za proceduralno programiranje te pokazivače za rad s memorijskim adresama, što omogućuje preciznu kontrolu toka programa i korištenje resursa računala. U kasnijim verzijama razvijene su i različite standardne biblioteke kojima je programerima olakšan rad s ulazno-izlaznim operacijama, matematičkim funkcijama i manipulacijom nizovima znakova.

Prednosti jezika C uključuju visoku brzinu izvođenja, široku i aktivnu zajednicu korisnika, dugogodišnju stabilnost te činjenicu da služi kao izvrsna polazna točka za učenje drugih programskih jezika. Budući da je na njemu zasnovan velik broj modernih jezika, razumijevanje C-a često se smatra osnovom programerskog obrazovanja. Osim u edukativne svrhe, C se i danas intenzivno koristi u industriji, osobito za razvoj operacijskih sustava, ugrađenih sustava (embedded systems), uređaja niske razine te softvera gdje su performanse od presudne važnosti.

Međutim, unatoč brojnim prednostima, C ima i određene nedostatke. Jezik ne posjeduje automatsko upravljanje memorijom, zbog čega je programer odgovoran za ručno alociranje i oslobađanje memorije. Zbog toga se lako mogu dogoditi pogreške poput curenja memorije (memory leak) ili pogrešnog pristupa adresama, koje često rezultiraju padom programa. Također, C nema ugrađenu podršku za objektno orijentirano programiranje, što ga čini manje pogodnim za moderne složene aplikacije koje se temelje na objektima. Upravo zbog tih ograničenja u novije vrijeme češće se koriste jezici višeg nivoa, no C i dalje ostaje neizostavan alat za razvoj sustava gdje su stabilnost, brzina i kontrola nad resursima ključni zahtjevi. [15]

4.2. Microsoft Visual Studio

Microsoft Visual Studio jedno je od najpoznatijih i najčešće korištenih integriranih razvojnih okruženja (IDE). Razvijen je od strane tvrtke Microsoft s ciljem da se programerima omogući jednostavnije i učinkovitije stvaranje različitih vrsta aplikacija. Okruženje je prvi put predstavljeno 1997. godine i od tada se kontinuirano razvija te nadopunjuje novim značajkama i alatima. Visual Studio podržava veliki broj programskih jezika, uključujući C, C++, C#, Python, JavaScript i mnoge druge, čime se omogućuje široka primjena u razvoju različitih softverskih rješenja.

U Visual Studiu naglasak je stavljen na jednostavnost korištenja i visoku produktivnost. Programerima su ponuđene napredne mogućnosti poput uređivanja i formatiranja koda, automatskog dovršavanja izraza, prijedloga pri pisanju, kao i integriranih alata za praćenje pogrešaka (debugging) i testiranje. Na taj način smanjuje se mogućnost nastanka pogrešaka i ubrzava proces razvoja. Osim toga, omogućeno je i povezivanje s bazama podataka, korištenje grafičkih sučelja za dizajniranje aplikacija, kao i integracija s različitim servisima u oblaku.

Posebna vrijednost Visual Studija prepoznata je u velikom broju ekstenzija i dodataka koji se mogu instalirati prema potrebama korisnika. Time se razvojno okruženje može prilagoditi specifičnim zahtjevima projekta i osobnim preferencijama programera. Vizualno sučelje omogućuje pregledno upravljanje datotekama, projektima i resursima, što uvelike olakšava rad u većim timovima. Također, integrirana je podrška za verzioniranje koda putem alata kao što su Git i GitHub, čime se pojednostavljuje suradnja među više programera.

Velika prednost Visual Studija ogleda se i u njegovoj fleksibilnosti. Dostupne su različite verzije, uključujući Visual Studio Community, koji je besplatan za studente, individualne programere i

manje projekte, kao i profesionalne i enterprise verzije namijenjene većim tvrtkama i složenim sustavima. Time je osigurano da razvojno okruženje može koristiti širok spektar korisnika, od početnika do iskusnih stručnjaka.

Visual Studio koristi se u industriji, obrazovanju i istraživanju te je kroz godine postao standardno razvojno okruženje za mnoge programere diljem svijeta. Zahvaljujući stalnim nadogradnjama i velikoj zajednici korisnika, osigurano je da ovo okruženje ostane jedno od najnaprednijih alata za razvoj softvera. [16]

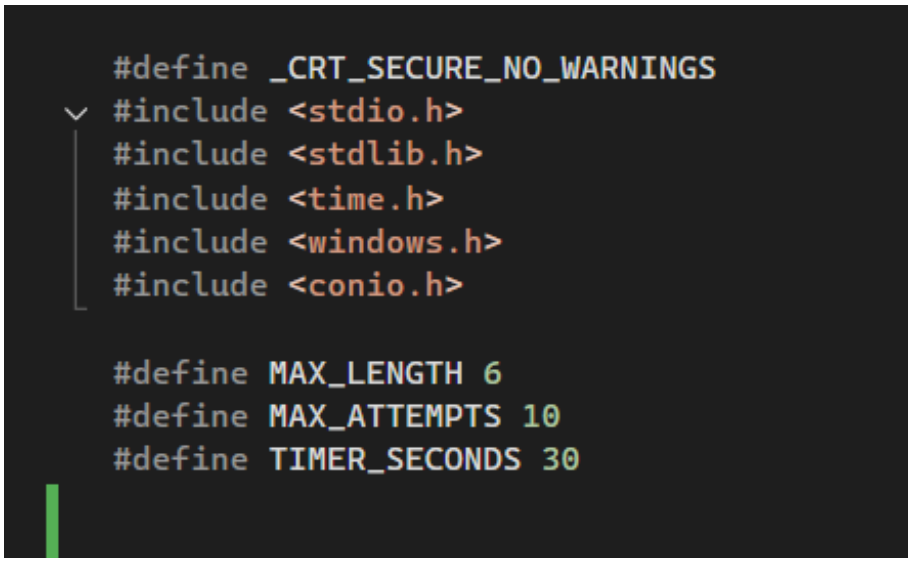
5. PROGRAMSKO RJEŠENJE

Program prikazan u nastavku ovog rada predstavlja detaljnu implementaciju popularne logičke igre Mastermind, pri čemu je u ovoj verziji uvedena značajna prilagodba jer se umjesto klasičnih boja koriste znamenke. U numeričkoj varijanti igre potrebno je pogoditi tajni niz brojeva koji se nasumično generira pri svakom pokretanju igre. Svaki unos uspoređuje se s tajnim nizom, a pritom se daje povratna informacija o tome koje su znamenke točne i nalaze li se na pravoj poziciji, odnosno koje su prisutne u nizu, ali na pogrešnom mjestu.

Posebnost ove implementacije leži u dodatnim elementima koji igru čine vizualno privlačnijom. U igru je implementirano vremensko ograničenje, što igraču stvara osjećaj pritiska i prisiljava ga na brzo donošenje odluka. Vremensko ograničenje od 30 sekundi po pokušaju osigurava dinamičnost igre i sprječava beskonačno promišljanje. Nadalje, korištenje boja u konzoli služi kao vizualna povratna informacija: zelena boja označava znamenke koje su točne i na ispravnoj poziciji, narančasta označava znamenke koje se nalaze u nizu, ali nisu na pravoj poziciji, dok bijela označava pogrešne znamenke koje nisu prisutne u tajnom nizu.

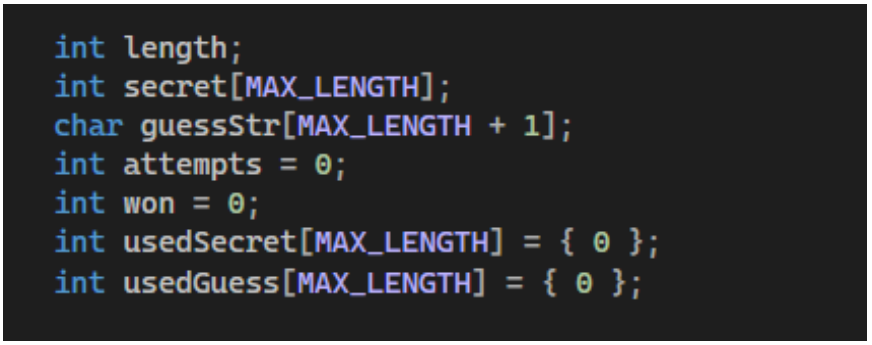
5.1. Varijable i konstante

Za početak, nužno je razumijevanje varijabli i konstanti korištenih u igri. Na samom vrhu programa definirane su konstante pomoću direktive *#define*. Konstanta *MAX_LENGTH* određuje najveću moguću duljinu niza koji se može pogađati, a u ovom slučaju ta duljina je postavljena na šest znamenki. Sljedeća konstanta, *MAX_ATTEMPTS*, određuje koliko puta je dopušteno pokušati pogoditi tajni broj, a ta vrijednost je deset. Na taj način igra je ograničena i broj pokušaja nije neograničen. Treća konstanta je *TIMER_SECONDS*, koja određuje maksimalno trajanje jedne runde, odnosno koliko sekundi je na raspolaganju da bude unesen pokušaj. U ovom programu radi se o trideset sekundi. Sve tri konstante omogućuju fleksibilnost igre jer ih je moguće mijenjati na jednom mjestu u programu bez potrebe da se prepravlja ostatak programskog koda. Definiranje gore navedenih konstanti prikazano je slikom 5.1.



Slika 5.1 : Konstante programa

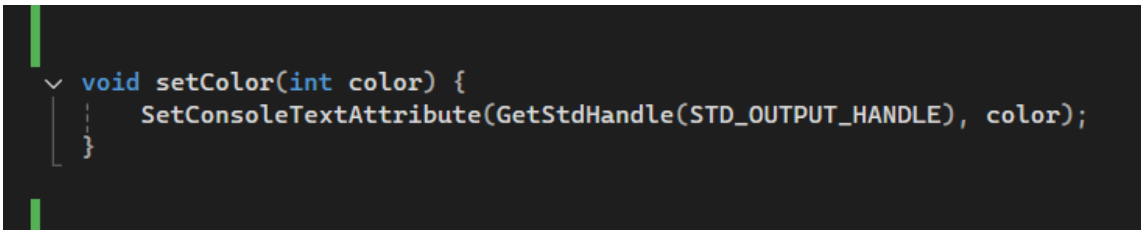
Osim konstanti, u programu se koriste i niz varijabli, prikazanih na slici 5.2, koje služe za kontrolu tijeka igre. Najvažnija varijabla na početku je *length*, koja predstavlja duljinu tajnog niza, a unosi se prilikom pokretanja programa. Zatim postoji niz *secret* u kojem se pohranjuju nasumično odabrane znamenke koje čine tajni niz. Taj niz generira se računalom i čuva u memoriji sve dok igra ne bude završena. Unos od strane korisnika pohranjen je u varijablu tipa *string* pod nazivom *guessStr*, koja može sadržavati maksimalno onoliko znakova koliko je definirano konstantom *MAX_LENGTH*. Dodatno, varijabla *attempts* koristi se za brojanje iskorištenih pokušaja, dok se varijablom *won* prati je li pogođen cijeli niz. Unutar svake runde koriste se i pomoćni nizovi *usedSecret* i *usedGuess* kojima se prati koje su znamenke već uspoređene i označene kao pogođene, čime se izbjegava višestruko brojanje iste znamenke.



Slika 5.2 : Varijable programa

5.2. Pomoćne funkcije

Posebno zanimljiv dio programa čine funkcije koje nadopunjuju osnovnu logiku igre i daju joj dodatne mogućnosti. Funkcija *setColor* koristi se za mijenjanje boje teksta unutar konzole. Zahvaljujući njoj, omogućeno je vizualno razlikovanje točnih znamenki, djelomično pogođenih znamenki i onih koje nisu uopće prisutne u tajnom nizu. Funkciji se predaje parametar *color* tipa *int*. *SetConsoleTextAttribute* predstavlja ugrađenu Windows funkciju kojom je omogućen ispis znakova na konzolu u boji koja se zahtijeva. Funkcija *setColor* služi kao "omot" za ugradbenu funkciju i omogućuje lakše mijenjanje boja u daljnjem programskom kodu. Ova funkcija prikazana je slikom 5.3.



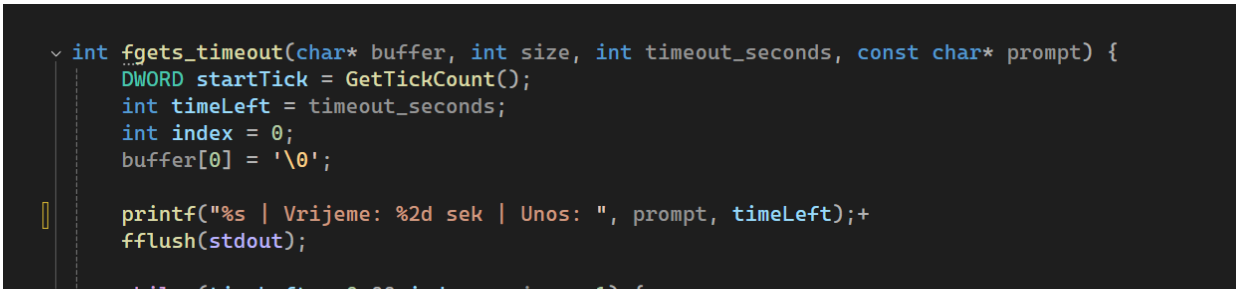
Slika 5.3 : Funkcija setColor()

Funkcija *fgets_timeout* (slika 5.4) daleko je složenija i omogućuje da unos niza bude vremenski ograničen. U njoj se koriste sistemske funkcije Windows okruženja kako bi proteklo vrijeme bilo mjereno i istodobno omogućen unos znamenki. Ako cijeli pokušaj ne bude unesen unutar predviđenog vremena, unos se prekida i prikazuje se obavijest da je isteklo vrijeme. Funkcija koristi četiri različita parametra. Parametar *buffer* predstavlja niz znakova u koji se sprema unos, *size* označava maksimalnu veličinu tog niza, *timeout_seconds* određuje vrijeme koje je na raspolaganju za unos podataka, odnosno niza, dok parametar *prompt* prikazuje poruku koja se ispisuje korisniku. Povratna vrijednost može biti dva broja:

- 1 : ako je korisnik unio podatke i pritisnuo Enter na vrijeme
- 0 : ako je isteklo vrijeme prije unosa

U sljedećim linijama programskog koda definirani su dodatni atributi. Atribut *startTick* tipa je *DWORD*, odnosno 32-bitni *unsigned integer*. U njega se sprema *GetTickCount()* kojim se vraća vrijeme u milisekundama od pokretanja Windowsa. *Index* služi za praćenje broja znakova koji su uneseni.

Prije početka glavne petlje za unos ispisuje se poruka korisniku. U njoj se ispisuje broj pokušaja koji je već odigran te brojač vremena od 30 sekundi. Funkcija *fflush* osigurava da se poruka odmah prikaže na ekranu.

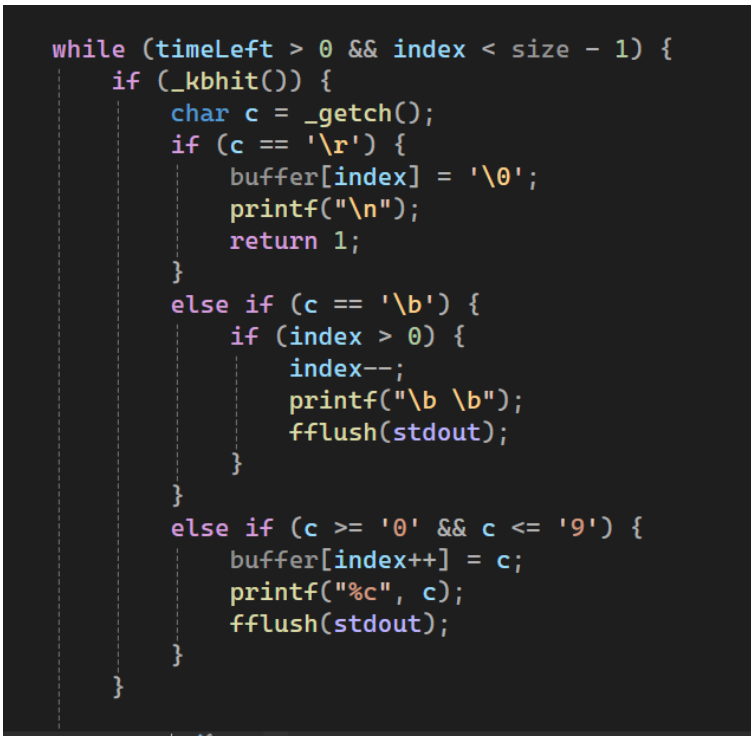


```
int fgets_timeout(char* buffer, int size, int timeout_seconds, const char* prompt) {
    DWORD startTick = GetTickCount();
    int timeLeft = timeout_seconds;
    int index = 0;
    buffer[0] = '\0';

    printf("%s | Vrijeme: %2d sek | Unos: ", prompt, timeLeft);+
    fflush(stdout);
```

Slika 5.4 : Početni segment main funkcije

Unutar glavne petlje provjerava se da vrijeme za unos nije isteklo i da unos nije pun, odnosno da je ostavljen prostor za *\0*. Ukoliko su oba uvjeta zadovoljena, ulazi se u petlju. Pomoću *kbhit()* provjerava se je li pritisnuta neka tipka. Ako jest, znak se odmah dohvaća i sprema u varijablu *c*. Zatim se provjerava koja tipka je pritisnuta. Ako je pritisnuta tipka Enter (*\r*), dodaje se završni znak kako bi string bio ispravan, prelazi se u novi red i vraća vrijednost 1, čime se označava da je unos uspješan. U drugom slučaju, ukoliko je pritisnut Backspace (*\b*), potrebno je izbrisati prethodni znak. *Index* briše znak iz *buffer*-a, dok *printf* vizualno briše znak iz konzole. U zadnjoj *if* petlji provjerava se točnost unosa, odnosno je li uneseni znak broj. Ako je broj unesen, sprema se u *buffer* i ispisuje na ekran. Zatim se pomiče pokazivač na sljedeću poziciju i izlazi iz petlje. Ovaj dio programskog koda prikazan je slikom 5.5.



```
while (timeLeft > 0 && index < size - 1) {
    if (_kbhit()) {
        char c = _getch();
        if (c == '\r') {
            buffer[index] = '\0';
            printf("\n");
            return 1;
        }
        else if (c == '\b') {
            if (index > 0) {
                index--;
                printf("\b \b");
                fflush(stdout);
            }
        }
        else if (c >= '0' && c <= '9') {
            buffer[index++] = c;
            printf("%c", c);
            fflush(stdout);
        }
    }
```

Slika 5.5 : Provjera pritisnute tipke

Ostatak funkcije koristi se za praćenje vremena preostalog za unos pokušaja. Za varijablu *elapsed* računa se koliko je sekundi proteklo od zadnjeg ispisa. *If* petljom ograničeno je da se ažurira prikaz vremena samo kada je prošla barem jedna sekunda. Varijabla *timeLeft* predstavlja preostalo vrijeme za unos, a od nje se oduzimaju protekle sekunde. Na kraju je potrebno promijeniti vrijednost varijable *startTick*. Njezina vrijednost postavlja se na vrijednost trenutno spremljenu u varijabli *elapsed*. Time je omogućeno da u idućoj iteraciji *elapsed* ne uključuje sve protekle milisekunde, nego broji samo novo proteklo vrijeme. Ostatak programskog koda omogućuje ažuriranje ispisa u konzoli svake sekunde s ispisom svih dosad unesenih znakova. Dio programskog koda za praćenje vremena preostalog za unos pokušaja prikazan je slikom 5.6..

```
        DWORD now = GetTickCount();
        int elapsed = (int)((now - startTick) / 1000);
        if (elapsed >= 1) {
            timeLeft -= elapsed;
            if (timeLeft < 0) timeLeft = 0;
            startTick += elapsed * 1000;

            printf("\r%s | Vrijeme: %2d sek | Unos: ", prompt, timeLeft);
            for (int i = 0; i < index; i++) {
                putchar(buffer[i]);
            }
            fflush(stdout);
        }
        Sleep(50);
    }
```

Slika 5.6 : Praćenje vremena za unos pokušaja

Još jedna dodatna funkcija je *countdown*, koja se koristi za kratko odbrojavanje prije početka svake nove runde. Kada jedan pokušaj bude završen, prije nego što započne novi, na ekranu se prikazuje odbrojavanje od nekoliko sekundi. Time se simulira mali prijelaz između pokušaja i omogućuje trenutak pripreme. Funkciji se predaje parametar *sec* koji predstavlja broj sekundi za odbrojavanje. Petlja funkcije počinje od *sec* i smanjuje se za 1 sekundu dok ne dosegne 0. Unutar *printf* pomoću *%d* prikazuje se trenutna vrijednost odbrojavanja. Funkcija *Sleep* osigurava zaustavljanje izvršavanja i omogućuje da odbrojavanje bude vidljivo u stvarnom vremenu. U ovom primjeru ono iznosi tisuću milisekundi, odnosno jednu sekundu. Implementacija funkcije prikazana je slikom 5.7.

```
void countdown(int sec) {
    for (int i = sec; i > 0; i--) {
        printf("\rNova runda pocinje za %d...", i);
        fflush(stdout);
        Sleep(1000);
    }
    printf("\r%*s\r", 30, "");
}
```

Slika 5.7 : Funkcija countdown()

5.3. Glavni dio programa

Središnji dio programa nalazi se u glavnoj funkciji *main*. Na početku se ispisuje poruka dobrodošlice i objašnjava pravilo da je potrebno unijeti proizvoljnu duljinu niza. Kada je broj unesen, na temelju njega generira se tajni niz znamenki pomoću funkcije *rand()*. Unutar funkcije osigurano je da se mogu nalaziti samo znamenke između 0 i 9. Nakon toga započinje glavni ciklus igre koji traje sve dok nisu potrošeni svi pokušaji ili dok cijeli niz ne bude pogođen. Opisani dio programskog koda prikazan je slikom 5.8.

```c
int main() {
    srand(time(NULL));
    int length;
    printf("Dobrodosli u Mastermind (brojevi)\n");
    printf("Unesi duljinu niza (1-%d): ", MAX_LENGTH);
    scanf_s("%d", &length);

    int secret[MAX_LENGTH];
    for (int i = 0; i < length; i++) {
        secret[i] = rand() % 10;
    }
```

Slika 5.8 : Početni segment main funkcije

Sada se objašnjava glavna petlja igre. *While* petlja izvodi se sve dok je broj pokušaja manji od maksimalnog broja pokušaja i dok tajni niz nije pogođen. Svaki puta kada započne nova runda, brojač pokušaja povećava se za jedan, a radi preglednosti ispisuje se separator. Poziva se funkcija *countdown* s argumentom 2 kojom se na početku svakog novog pokušaja omogućuje odbrojavanje. Nadalje, koristeći varijablu *attempts* definira se string koji se prosljeđuje u funkciju *fgets_timeout*. Svakom novom iteracijom unutar stringa mijenja se broj pokušaja za pogađanje tajnog niza.

Definirana je varijabla *inputOK* u koju se sprema vrijednost funkcije *fgets_timeout* kako bi bilo provjereno je li niz unesen u zadanom vremenu. Ukoliko vrijeme istekne, odnosno kada *inputOK* ima vrijednost nula, boja teksta postavlja se u crveno i ispisuje se odgovarajuća poruka. Nakon toga potrebno je vratiti boju teksta u bijelu te se ostatak petlje preskače i prelazi na novi pokušaj. Opisani dio programskog koda prikazan je slikom 5.9.

```
while (attempts < MAX_ATTEMPTS && !won) {
    attempts++;

    printf("\n----------------------------------------\n");
    countdown(2);

    char prompt[60];
    sprintf(prompt, "Pokusaj #%d/%d - Unesi %d znamenki", attempts, MAX_ATTEMPTS, length);

    int inputOK = fgets_timeout(guessStr, sizeof(guessStr), TIMER_SECONDS, prompt);
    if (!inputOK) {
        setColor(12);
        printf("Isteklo vrijeme!\n");
        setColor(7);
        continue;
    }
```

Slika 5.9 : Provjera točnog unosa

Također je potrebno provjeriti koliko je znamenki uneseno. To se provjerava jednostavnom *if* petljom u kojoj se uspoređuje duljina unesenog niza i duljina niza koji se pogađa. Ukoliko taj broj nije jednak, ispisuje se poruka i izlazi se iz petlje. Time se gubi trenutni pokušaj i prelazi na idući. Petlja je prikazana slikom 5.10.

```
    if (strlen(guessStr) != length) {
        setColor(12);
        printf("Morate unijeti %d znamenki!\n", length);
        setColor(7);
        continue;
    }
```

Slika 5.10 : Provjera unosa točne duljine niza

Zatim slijedi provjera i usporedba unesenog niza s tajnim nizom. U prvoj petlji provjerava se nalazi li se točna znamenka na točnom mjestu unutar niza. Unutar *if* petlje vidi se kako se od varijable *guessStr* oduzima vrijednost 0. Ova operacija izvodi se iz razloga što je unos korisnika tipa *char*, dok je varijabla *secret* tipa *int*. Kako bi bila omogućena usporedba ove dvije varijable, *guessStr[i]* mora biti pretvoren u varijablu tipa *int*. Ukoliko su ove dvije varijable istog iznosa, povećava se brojač točnih pozicija i označava da je ta pozicija iskorištena, odnosno da je znak postavljen na točnu poziciju. Također je potrebno provjeriti nalazi li se točan broj na krivom mjestu. Unutar ove petlje prolazi se kroz sve unesene znakove, ali provjeravaju se samo oni koji nisu označeni na pravom mjestu. Unutarnja petlja pretražuje sve pozicije tajnog niza kako bi pronašla istu znamenku na nekom drugom indeksu. Unutar *if* petlje moraju biti zadovoljena dva

uvjeta: pozicija u tajnom nizu ne smije biti već pogođena i znamenka se mora nalaziti unutar niza. Na kraju petlje unutarnja petlja mora biti prekinuta pomoću *break*. Time se osigurava da jedan znak ima najviše jedno podudaranje, odnosno da ne može biti povezan s više pozicija u tajnom nizu. Programski kod za provjeru niza prikazan je slikom 5.11.

```
int correctPos = 0, correctNum = 0;
int usedSecret[MAX_LENGTH] = {0}, usedGuess[MAX_LENGTH] = {0};

for (int i = 0; i < length; i++) {
    if (guessStr[i] - '0' == secret[i]) {
        correctPos++;
        usedSecret[i] = usedGuess[i] = 1;
    }
}

for (int i = 0; i < length; i++) {
    if (!usedGuess[i]) {
        for (int j = 0; j < length; j++) {
            if (!usedSecret[j] && guessStr[i] - '0' == secret[j]) {
                correctNum++;
                usedSecret[j] = 1;
                break;
            }
        }
    }
}
```

Slika 5.11 : Programski kod za provjeru niza

Kada je ova analiza završena, ispisuje se rezultat. To se izvodi tako da se kroz petlju najprije uspoređuju znamenke u tajnom nizu i nizu koji je unesen. Ako se ista znamenka nalazi u oba niza na istom mjestu, označava se zelenom bojom. Nakon toga provjerava se nalazi li se znamenka na drugoj poziciji. Ukoliko se nalazi, boja ispisa te znamenke mijenja se u narančastu. Na kraju je potrebno ispisati cijeli niz. Programski kod za ispis rezultata prikazan je slikom 5.12

```
printf("Rezultat: ");
for (int i = 0; i < length; i++) {
    if (guessStr[i] - '0' == secret[i]) {
        setColor(10);
    } else {
        int found = 0;
        for (int j = 0; j < length; j++) {
            if (guessStr[i] - '0' == secret[j]) {
                found = 1;
                break;
            }
        }
        if (found) setColor(14);
        else setColor(7);
    }
    printf("%c", guessStr[i]);
    setColor(7);
}
printf("\n");
```

Slika 5.12 : Ispis rezultata

Na kraju se uspoređuju duljina niza i broj točno pogođenih pozicija. Ukoliko je taj broj isti, na konzolu se ispisuje poruka o pobjedi igre u zelenoj boji. Atribut *won* potrebno je postaviti na 1 kako bi bilo spriječeno izvršavanje petlje kada tajni niz nije pogođen. Ispis za pogođeni niz prikazan je slikom 5.13.

```
if (correctPos == length) {
    setColor(10);
    printf("\nBravo! Pogodili ste tajni broj!\n");
    setColor(7);
    won = 1;
}
```

Slika 5.13 : Završetak igre u slučaju pogođenog niza

Ako cijeli niz na kraju ipak ne bude pogođen, ispisuje se poruka o završetku igre i prikazuje koji je bio tajni niz. Taj se ispis prikazuje crvenom bojom kako bi bio uočljiviji. Nakon toga program završava s radom. Ovaj dio programskog koda prikazan je slikom 5.14.

```
    if (!won) {
        setColor(12);
        printf("\nKraj igre! Tajni broj je bio: ");
        for (int i = 0; i < length; i++) {
            printf("%d", secret[i]);
        }
        printf("\n");
        setColor(7);
    }

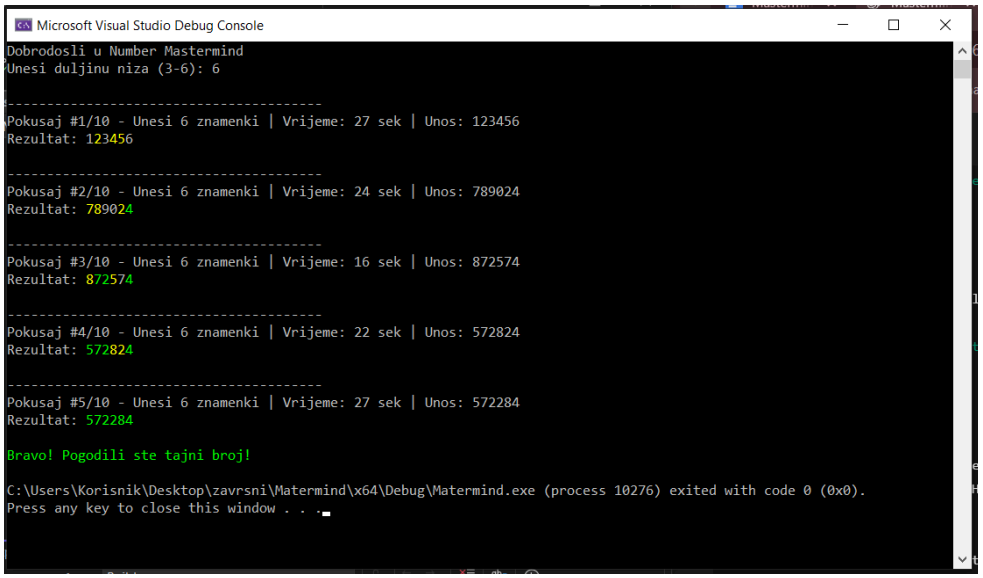
    return 0;
}
```

Slika 5.14 : Završetak igre u slučaju ne pogođenog niza

5.4. Prikaz rezultata

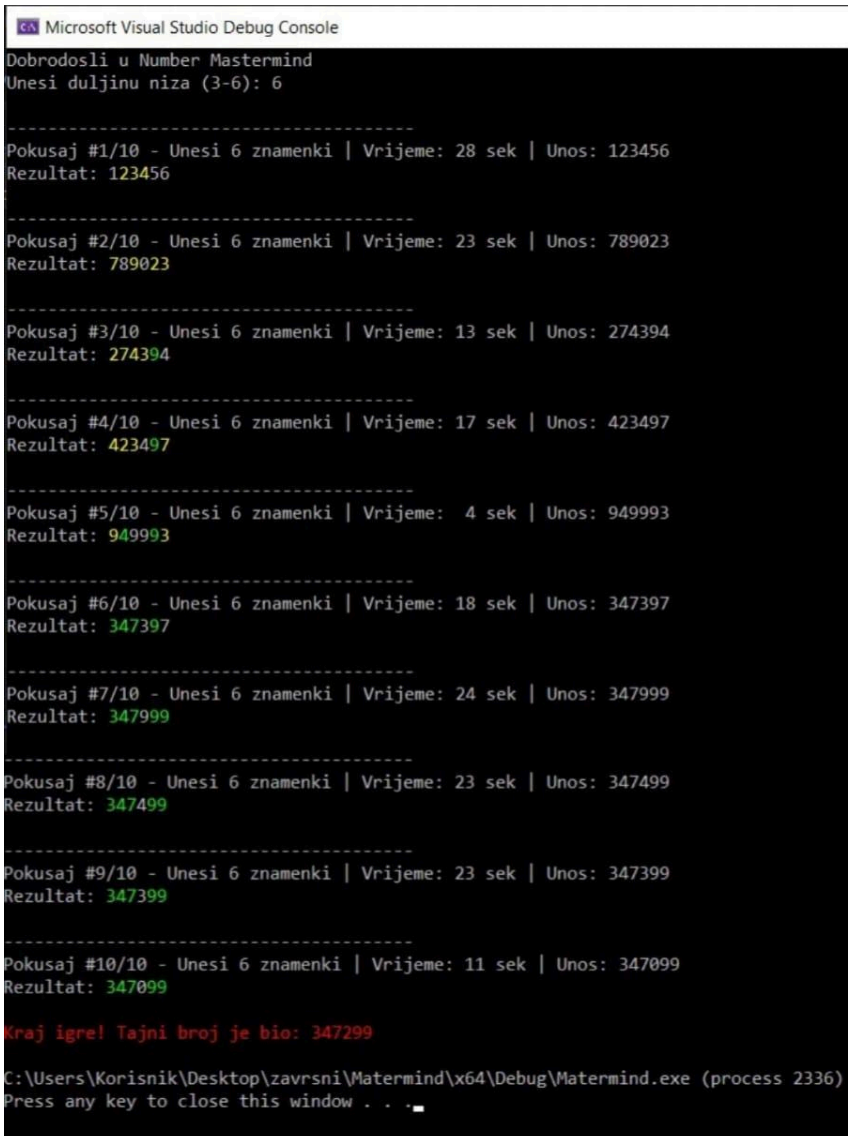
Kako bi se detaljnije prikazalo funkcioniranje aplikacije, u nastavku su predstavljeni primjeri odigranih igri. Analiza se temelji na četiri različita scenarija koja obuhvaćaju glavne mogućnosti i situacije koje se mogu pojaviti tijekom igre.

Na slici 5.15. prikazan je primjer igre u kojem je uspješno pogođen tajni niz duljine šest znamenki. Nakon pokretanja aplikacije najprije se prikazuje poruka i uputa da bude unesena duljina niza. Nakon unosa broja započinje se igra i prikazuje prvi redak u kojem stoji redni broj pokušaja, ukupan broj dostupnih pokušaja te preostalo vrijeme za unos. Nakon što je unesena kombinacija znamenki, na ekranu se prikazuje rezultat. Nakon svakog pokušaja jasno se prikazuju smjernice za daljnje pogađanje. Kada napokon bude unesen točan niz, cijela kombinacija prikazuje se u zelenoj boji, a ispod nje pojavljuje se poruka „Bravo! Pogodili ste tajni broj!” ispisana zelenim slovima. Time je jasno naznačeno da je igra uspješno završena.



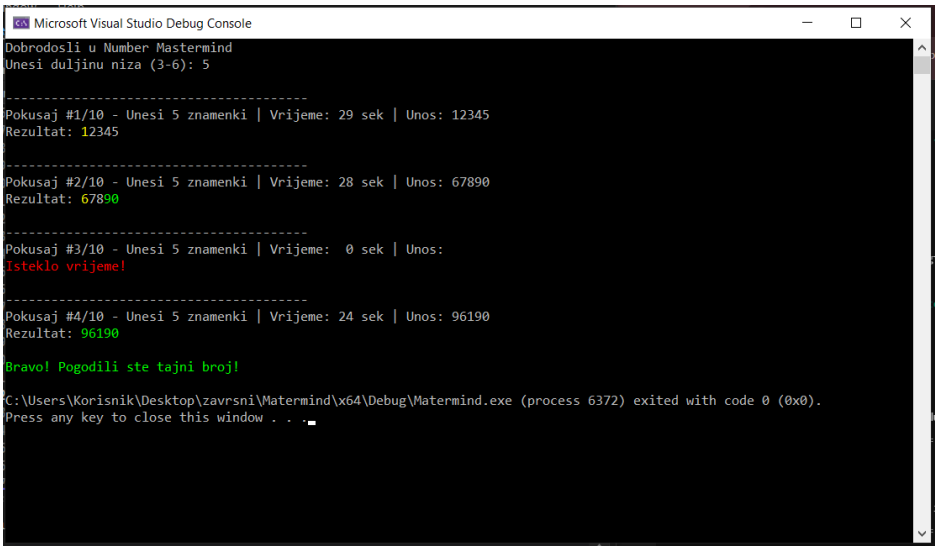
Slika 5.15 : Rezultat izvršavanja programa u slučaju pogođenog niza

Na slici 5.16. prikazan je primjer igre u kojem ni nakon deset pokušaja tajni niz nije pogođen. Tijekom svakog pokušaja prikazuje se redni broj poteza, preostalo vrijeme i unesena kombinacija, a zatim obojane znamenke koje pokazuju koliko je pokušaj bio uspješan. Nakon desetog pokušaja igra se završava i na ekranu se crvenom bojom ispisuje poruka „Kraj igre! Tajni broj je bio: …", pri čemu se otkriva tražena kombinacija. Na taj način jasno je dano do znanja da je igra završila neuspješno te je otkriveno ispravno rješenje koje je trebalo biti pogođeno. Ovaj scenarij pokazuje kako aplikacija pravilno obrađuje i završava igru u slučaju kada su potrošeni svi predviđeni pokušaji.



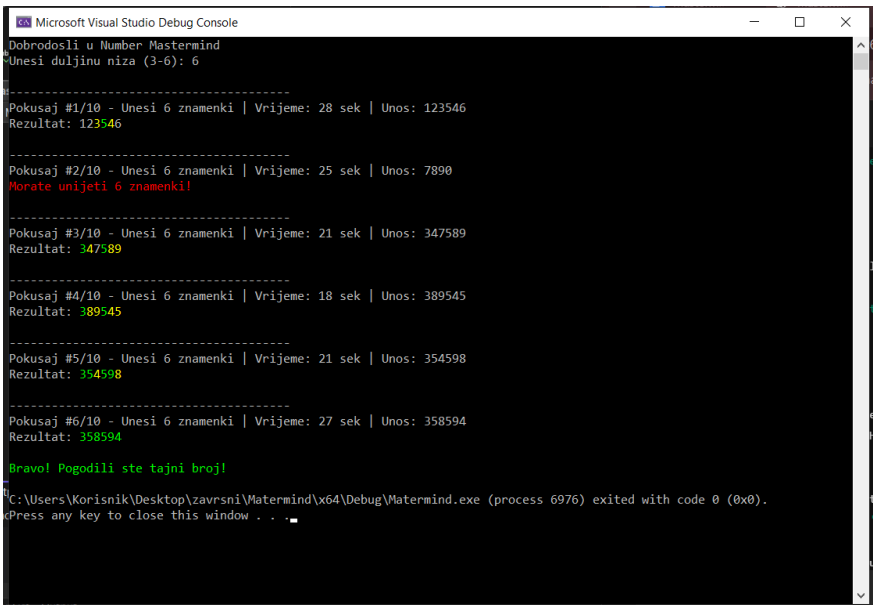
Slika 5.16 : Rezultat izvršavanja programa u slučaju ne pogođenog niza

Na slici 5.17. prikazana je situacija u kojoj kombinacija znamenki nije unesena unutar predviđenog vremenskog ograničenja od 30 sekundi. Nakon prvog i drugog pokušaja unesene vrijednosti normalno su prihvaćene i rezultat je prikazan u boji. Međutim, u trećem pokušaju ništa nije uneseno na tipkovnici, a vrijeme je isteklo. Program zatim automatski prekida čekanje i u konzoli crvenom bojom ispisuje poruku „Isteklo vrijeme!”. Taj pokušaj evidentiran je kao neuspješan, a odmah se prelazi na sljedeću priliku.



Slika 5.17 : Rezultat izvršavanja programa u slučaju istjecanja vremena

Na slici 5.18. prikazan je primjer u kojem tijekom igre nije unesen ispravan broj znamenki. Nakon što je u prvom pokušaju unesen valjani niz, u drugom pokušaju unesene su samo četiri znamenke umjesto šest. Program odmah reagira i na ekranu se ispisuje upozorenje crvenom bojom: „Morate unijeti 6 znamenki!”. Na taj način jasno je naznačeno da je došlo do pogreške u unosu, a pokušaj se ne računa kao valjan. Nakon toga igra se normalno nastavlja. Ovaj primjer pokazuje kako aplikacija učinkovito obrađuje pogrešan unos i pruža jasnu povratnu informaciju.



Slika 5.18 : Rezultat izvođenja programa u slučaju krivo unesenog niza

Analizom različitih scenarija može se uočiti da program pravilno reagira u svim predviđenim situacijama. Također je potvrđeno da je omogućeno osnovno igranje Masterminda te da se pogreške i posebne situacije pouzdano obrađuju, čime je osigurano jasno iskustvo igranja.

6. ZAKLJUČAK

Zadatak ovog završnog rada bio je razviti računalno rješenje za igru Mastermind u programskom jeziku C. Iako Mastermind na prvi pogled djeluje kao jednostavna društvena igra, njegova digitalna verzija otvara mogućnost primjene različitih algoritama i prikazuje kako se klasična logička zagonetka može prilagoditi računalnom okruženju. Postoje brojne inačice Mastermind-a, ali verzija temeljena na znamenkama, umjesto na bojama, manje je istražena i manje zastupljena u dostupnim programskim rješenjima. Upravo taj nedostatak programskih rješenja poslužio je kao poticaj za izradu ovog rada.

Implementirani program koristi nasumično generiranje tajnog niza znamenki te omogućuje korisniku da kroz ograničen broj pokušaja pogodi skriveni kod. Dodatno, program uključuje vremensko ograničenje za unos i vizualno označavanje rezultata pomoću boja. Rezultati pokretanja programa potvrđuju da se ovakav pristup može koristiti kao uspješna simulacija igre Mastermind, koja ujedno potiče deduktivno razmišljanje i strategijsko planiranje kod igrača. Time je pokazano da i relativno jednostavne logičke igre mogu poslužiti za razvoj programerskih vještina i praktičnu primjenu teorijskih znanja.

Mogućnosti za nadogradnju ovog rada su brojne. Program bi se mogao proširiti na različite duljine nizova, veći raspon znamenki ili dodavanje novih pravila koja bi povećala složenost igre. Također, mogla bi se razviti grafička verzija igre s vizualnim sučeljem, čime bi iskustvo igranja bilo još privlačnije.

Ovaj rad može poslužiti kao temelj za daljnji razvoj složenijih verzija Mastermind-a, uključujući proširenje za različita pravila, različite vrste i duljine niza, te razvoj grafičke igre s vizualnim sučeljem.

SAŽETAK

Naslov: Programsko rješenje logičke igre Mastermind u programskom jeziku C

Ovaj završni rad bavi se izradom programskog rješenja logičke igre Mastermind u programskom jeziku C. U radu su opisane razne vrste Mastermind igre, kao i Number Mastermind, algoritam rješavanja igre, te logička pozadina rješavanja ove igre. Opisan je način rješavanja Number Mastermind-a, te je kasnije razrađen i opisan programski kod korišten za izradu logičke zagonetke. Program generira niz brojeva kojega korisnik pokušava pogoditi, te vraća povratne informacije o pogotku na konzolu. Završni rad sadrži opise svih funkcija korištenih u programu i tehnologije koje su korištene.

Ključne riječi: C, logičke igre, Mastermind, Programsko rješenje,

ABSTRACT

Title: The Mastermind logic game in the C programming language

This thesis deals with the development of the Mastermind logic game software solution in the C programming language. The paper describes various types of Mastermind games, as well as the Number Mastermind, the algorithm for solving the game, and the logical background for solving this game. It describes the method of solving the Number Mastermind, and is later developed and describes the programming code used to construct the logic puzzle. The program generates a sequence of numbers that the user tries to guess, and returns the hit information back to the console. The final paper shall contain descriptions of all functions used in the programme and the technologies used.

Key words: C, Logic games, Mastermind, Software solution

LITERATURA

[1] D. E. Knuth, The computer as Master mind, Sveučilište Stanford, 1977., dostupno na: https://www.cs.uni.edu/~wallingf/teaching/cs3530/resources/knuth-mastermind.pdf

[2] N. Collina, Fastermind: Using a SAT-solver to play Mastermind more efficiently, 2018., dostupno na: https://www.seas.upenn.edu/~ncollina/Mastermind.pdf

[3] J.J. Merelo, P. Castillo, A. Mora, Improving Evolutionary Solutions to the Game of MasterMind Using an Entropy-based Scoring Method, Sveučilište u Granadi, 2013., dostupno na:http://www.cmap.polytechnique.fr/~nikolaus.hansen/proceedings/2013/GECCO/proceedings/p829.pdf

[4] Mastermind (board game), Wikipedia, dostupno na: https://en.wikipedia.org/wiki/Mastermind_(board_game) [21.6.2025.]

[5] Genetic algorithm, Wikipedia, dostupno na: https://en.wikipedia.org/wiki/Genetic_algorithm [1.9.2025.]

[6] A. D. Rhodes, Search algorithms for Mastermind, Sveučilište Portland, 2019., dostupno na: https://arxiv.org/pdf/1908.06183 [1.9.2025.]

[7] M. de Bondt, NP-completeness of Master Mind and Minesweeper, Sveučilište Radboud Nijmegen, Nizozemska, 2004., dostupno na: https://www.researchgate.net/publication/254888546_NP-completeness_of_Master_Mind_and_Minesweeper [3.9.2025.]

[8] 1-in-3-sat, Wikipedia, dostupno na: https://en.wikipedia.org/wiki/1-in-3-SAT [1.9.2025.]

[9] J. Stuckman, G.Q. Zhang, Mastermind is NP-Complete, Sveučilište Case Western Reserve, SAD, 2005., dostupno na: https://arxiv.org/pdf/cs/0512049 [20.8.2025.]

[10] EasySurf.cc, Mastermind igra, dostupno na: https://www.easysurf.cc/master3.htm [16.9.2025.]

[11] dcode.fr, Mastermind solver, dostupno na: https://www.dcode.fr/mastermind-solver [16.9.2025.]

[12] Chkoupinator, Mastermind Numbers Solver, 2021., dostupno na: https://gitlab.com/chkoupinator/mastermind-solver [16.9.2025.]

[13] gnwong, mastermind-solver, dostupno na: https://github.com/gnwong/mastermind-solver [16.9.2025.]

[14] SaadARazzaq, Mastermind Game, dostupno na: https://github.com/SaadARazzaq/Mastermind-Game [16.9.2025.]

[15] Visual Studio, Wikipedia, dostupno na: https://en.wikipedia.org/wiki/Visual_Studio [21.6.2025.]

[16] C (programski jezik), Wikipedia, dostupno na: https://hr.wikipedia.org/wiki/C_(programski_jezik) [21.6.2025.]